

CSE232 시스템프로그래밍(캡스톤디자인)

4주차 첫 번째 수업

리눅스 파일과 디렉토리

경민기

수업 구성

1부 파일 이름, inode와 경로 · 15분

2부 파일 속성과 접근 권한 · 25분

3부 하드 링크와 심볼릭 링크 · 25분

4부 디렉토리 탐색과 관리 · 30분

확인 퀴즈와 실습 정리 · 15분

120분 수업 · 2부 후 휴식 10분

1부

파일과 경로

파일 이름, inode, 데이터의 관계를 살펴봅니다.

파일 이름, inode와 데이터

디렉토리 엔트리는 파일 이름과 inode 번호를 연결한다.

inode에는 파일 종류, 권한, 소유자 등의 정보가 있다.

일반 파일의 내용은 파일시스템이 관리하는 바이트다.

같은 파일을 여러 이름으로 참조할 수 있다.

Linux의 파일 종류

- 일반 파일: 텍스트, 이미지, 실행 파일

d 디렉토리: 파일 이름과 대상을 연결

l 심볼릭 링크: 다른 대상을 가리키는 경로

c / b 문자 장치와 블록 장치

p / s FIFO와 소켓

절대 경로와 상대 경로

```
/home/student/work/note.txt
```

```
# 현재 위치: /home/student/work  
./note.txt  
../work/note.txt
```

```
pwd  
ls -ld . ..  
ls -l ./note.txt
```

절대 경로

/에서 시작한다.

상대 경로

현재 작업 디렉토리 등을
기준으로 해석한다.

특수 이름

.은 현재 디렉토리,
..은 부모 디렉토리다.

2부

파일 속성과 권한

ls와 stat의 정보를 C 프로그램에서 읽습니다.

ls -l 출력 읽기

```
$ ls -l note.txt
-rw-r----- 2 student lab 4 Sep 21 10:00 note.txt

# 유형          -
# 권한          rw- r-- ---
# 링크 수       2
# 소유자        student
# 그룹          lab
# 크기          4 bytes
```

종류와 권한

첫 문자와 다음 9개 문자를 나누어 읽는다.

소유권

사용자와 그룹별로 권한을 구분한다.

크기

일반 파일의 크기는 내용의 바이트 수다.

출력 예시: 계정, 그룹, 시각은 실행 환경에 따라 달라집니다.

stat, lstat, fstat

stat(path, &st)

경로가 가리키는 파일의 속성을 읽는다. 마지막 심볼릭 링크를 따라간다.

lstat(path, &st)

마지막 경로 요소가 심볼릭 링크이면 링크 자체의 속성을 읽는다.

fstat(fd, &st)

이미 열어 둔 파일 디스크립터로 속성을 읽는다.

공통 헤더: <sys/stat.h> · 성공 0, 실패 -1

struct stat의 주요 필드

st_dev, st_ino: 파일시스템 식별 정보와 inode 번호

st_mode: 파일 종류와 접근 권한 비트

st_nlink: 하드 링크 수 st_uid, st_gid: 소유자

st_size: 일반 파일의 바이트 크기

st_atime / st_mtime / st_ctime: 접근 / 내용 수정 / 상태 변경

ctime은 생성 시각이 아니라 inode 상태 변경 시각입니다.

mystat: 속성 조회와 파일 종류

```
struct stat st;
if (lstat(argv[1], &st) == -1) {
    perror(argv[1]);
    return 1;
}

if (S_ISREG(st.st_mode)) puts("regular");
else if (S_ISDIR(st.st_mode)) puts("directory");
else if (S_ISLNK(st.st_mode)) puts("symlink");

printf("inode=%ju links=%ju\n",
        (uintmax_t)st.st_ino,
        (uintmax_t)st.st_nlink);
```

호출 결과

조회에 성공한 뒤에만
st의 필드를 사용한다.

종류 판정

S_ISREG, S_ISDIR, S_ISLNK
매크로를 사용한다.

출력 형식

정수형을 uintmax_t로 변환해
%ju로 출력한다.

접근 권한 비트

소유자(u), 그룹(g), 기타 사용자(o)에 각각 rwx를 부여한다.

일반 파일의 r은 읽기, w는 쓰기, x는 실행 권한이다.

r = 4, w = 2, x = 1을 더해 8진수로 나타낸다.

0640 = rw-r----- 0755 = rwxr-xr-x

st_mode & 0777로 기본 rwx 비트만 추출한다.

디렉토리의 r, w, x

r: 이름 목록 읽기

디렉토리 안의 파일 이름을 나열할 수 있다.

x: 이름 검색과 경로 통과

이미 아는 이름으로 내부 객체를 찾고 경로를 따라갈 수 있다.

w: 디렉토리 엔트리 변경

이름 생성·삭제에는 일반적으로 디렉토리의 w와 x가 함께 필요하다.

일반 사용자와 기본 권한을 기준으로 관찰합니다.

chmod와 umask

```
# 기존 파일의 권한 변경
chmod 640 note.txt
ls -l note.txt

# C API: 성공 0, 실패 -1
if (chmod("note.txt", 0640) == -1)
    perror("chmod");

# 새 파일의 생성 권한 예시
0666 & ~0022 = 0644
```

chmod

기존 파일의 권한을 바꾼다.
반환값을 확인한다.

umask

새 객체 생성 시
허용하지 않을 비트를 제거한다.

8진수 표기

C의 0640과 십진수 640을
구분한다.

생성 권한 계산은 부모 디렉토리에 기본 ACL이 없는 경우입니다.

실습 1: 파일 속성 확인

```
RUN=$(mktemp -d)
printf 'ABC\n' > "$RUN/note.txt"
mkdir "$RUN/sub"
```

```
./mystat "$RUN/note.txt"
./mystat "$RUN/sub"
chmod 640 "$RUN/note.txt"
./mystat "$RUN/note.txt"
./mystat "$RUN/missing"
```

내용과 크기

ABC와 개행을 합쳐
4바이트인지 확인한다.

종류와 권한

regular / directory를 구별하고
권한 변경 전후를 비교한다.

실패 경로

없는 경로의 오류 메시지와
비정상 종료 상태를 확인한다.

이후 실습도 같은 셸의 RUN 디렉토리에서 진행합니다.

3부

하드 링크와 심볼릭 링크

같은 파일에 붙는 이름과 경로를 저장하는 링크를 비교합니다.

하드 링크

같은 파일의 다른 이름

여러 디렉토리 엔트리가 같은 inode를 가리킨다.

관찰할 값

같은 파일시스템에서 `st_ino`가 같고 `st_nlink`가 증가한다.

생성 범위

서로 다른 파일시스템 사이에는 만들 수 없다. 디렉토리는 일반적으로 제한한다.

명령: `ln old.txt new.txt` C API: `link(oldpath, newpath)`

심볼릭 링크

다른 경로를 저장하는 파일

대상을 찾을 때 저장된 경로를 해석한다. 별도의 inode를 가진다.

대상과 수명이 분리됨

대상이 없어도 링크는 존재할 수 있다. 디렉토리나 다른 파일시스템도 가리킨다.

상대 경로의 기준

상대 대상 경로는 링크가 들어 있는 디렉토리를 기준으로 해석한다.

명령: `ln -s target.txt shortcut.txt` C API: `symlink(target, linkpath)`

심볼릭 링크에서 stat과 lstat

```
# links.txt는 original.txt를 가리키는 symlink
stat("links.txt", &st);
/* original.txt의 속성 */

lstat("links.txt", &st);
/* links.txt 자체의 속성 */

# original.txt가 없어진 뒤
stat("links.txt", &st); /* -1, ENOENT */
lstat("links.txt", &st); /* 링크는 조회 가능 */
```

stat

대상 파일을 따라가므로
대상이 없으면 실패한다.

lstat

링크 자체를 조회한다.
S_ISLNK로 판정할 수 있다.

오류 처리

실제 코드에서는 모든 호출의
반환값을 검사한다.

깨진 링크: 링크 파일은 남아 있으나 대상 경로를 해석할 수 없는 상태

실습 2: 이름을 삭제한 뒤 비교

```
ln "$RUN/note.txt" "$RUN/hard.txt"
ln -s note.txt "$RUN/soft.txt"

ls -li "$RUN"
./mystat "$RUN/hard.txt"
./mystat "$RUN/soft.txt"

rm "$RUN/note.txt"
cat "$RUN/hard.txt"
cat "$RUN/soft.txt"
./mystat "$RUN/soft.txt"
```

삭제 전

note.txt와 hard.txt의 inode, 링크 수를 비교한다.

삭제 후

hard.txt로 내용을 읽을 수 있고 soft.txt는 깨진 링크가 된다.

링크 자체

mystat은 lstat을 사용하므로 soft.txt의 속성은 읽을 수 있다.

실습 1에서 생성한 RUN 디렉토리 안에서만 실행합니다.

unlink와 열린 파일의 수명

unlink는 디렉토리에서 이름 하나를 제거한다.

다른 하드 링크가 남아 있으면 다른 이름으로 접근할 수 있다.

마지막 이름을 지워도 이미 열린 FD로 파일을 사용할 수 있다.

마지막 하드 링크와 열린 참조가 사라지면 저장 공간을 반환할 수 있다.

4부

디렉토리 탐색과 관리

디렉토리의 이름 목록을 읽고 각 파일의 속성을 조회합니다.

디렉토리 스트림

opendir(path)

디렉토리를 열어 DIR *를 얻는다. 실패하면 NULL을 반환한다.

readdir(dir)

다음 엔트리를 가리키는 struct dirent *를 얻는다.

closedir(dir)

디렉토리 스트림을 닫는다. 성공 0, 실패 -1이다.

헤더: <dirent.h> · readdir의 NULL은 끝 또는 오류입니다.

readdir의 끝과 오류 구분

```
for (;;) {
    errno = 0;
    struct dirent *ent = readdir(dir);
    if (ent == NULL) {
        if (errno != 0) {
            perror("readdir");
            status = 1;
        }
        break;
    }
    /* ent->d_name을 처리한다. */
}
if (closedir(dir) == -1) {
    perror("closedir"); status = 1;
}
```

매번 초기화

readdir를 호출하기 직전에
errno를 0으로 만든다.

NULL 판별

errno가 0이면 끝,
0이 아니면 오류다.

정리

반복문을 끝낸 뒤에는
closedir를 호출한다.

디렉토리 엔트리 처리

d_name은 디렉토리 안의 이름이다. 전체 경로가 아니다.

다음 readdir 호출 전에 필요한 이름과 정보를 사용하거나 복사한다.

반환된 포인터를 free하지 않는다.

d_type이 DT_UNKNOWN일 수 있으므로 속성 조회를 준비한다.

출력 순서는 정렬되지 않을 수 있다. .과 ..의 처리 정책을 정한다.

엔트리의 속성 조회

```
struct stat st;
int dfd = dirfd(dir);
if (dfd == -1) {
    perror("dirfd");
    closedir(dir); /* 정리 후 실패 반환 */
    return 1;
}

if (fstatat(dfd, ent->d_name, &st,
            AT_SYMLINK_NOFOLLOW) == -1) {
    perror(ent->d_name);
    status = 1;
    continue;
}
```

dirfd는 반복 전에, fstatat은 반복 안에서 호출합니다.

기준 디렉토리

열린 디렉토리의 FD를
기준으로 이름을 찾는다.

링크 정책

AT_SYMLINK_NOFOLLOW로
마지막 symlink 자체를 조회한다.

개별 실패

실패한 이름을 기록하고
다음 엔트리를 읽는다.

디렉토리 생성과 삭제

mkdir(path, mode)

새 디렉토리를 만든다. 생성 권한에는 umask 등의 조건이 적용된다.

rmdir(path)

비어 있는 디렉토리를 삭제한다. 안에 항목이 있으면 실패한다.

결과 확인

두 함수 모두 성공 0, 실패 -1이다. errno로 실패 원인을 확인한다.

헤더: mkdir는 <sys/stat.h>, rmdir는 <unistd.h>

현재 작업 디렉토리

```
char cwd[4096];

if (getcwd(cwd, sizeof cwd) == NULL) {
    perror("getcwd");
    return 1;
}
puts(cwd);

if (chdir("sub") == -1) {
    perror("chdir");
    return 1;
}
```

헤더: <unistd.h> · 경로 버퍼가 부족한 경우도 오류로 처리합니다.

getcwd

현재 작업 디렉토리의
절대 경로를 얻는다.

chdir

현재 프로세스의
작업 디렉토리를 바꾼다.

영향 범위

이후 상대 경로의 기준이
바뀐다. 부모 셸은 바뀌지 않는다.

실습 3: 작은 ls 실행

```
touch "$RUN/.hidden"
printf 'hello\n' > "$RUN/space name.txt"

./mini_ls "$RUN"
./mini_ls "$RUN/sub"
./mini_ls "$RUN/space name.txt"
./mini_ls "$RUN/missing"

# Ubuntu에서 시스템 호출 관찰
strace -e openat,getdents64,newfstatat,close \
  ./mini_ls "$RUN"
```

정상 입력

하위 이름, 종류, 크기를
출력하는지 확인한다.

경계 입력

빈 디렉토리, 숨김 파일,
공백이 있는 이름을 확인한다.

실패 입력

일반 파일과 없는 경로를
인자로 주어 오류를 확인한다.

출력 순서와 inode 번호는 환경에 따라 달라집니다.

확인 퀴즈

하드 링크 두 개는 inode와 파일 내용을 어떻게 공유하는가?

깨진 심볼릭 링크에 stat과 lstat을 호출하면 어떻게 다른가?

디렉토리의 x 권한은 어떤 동작에 필요한가?

readdir가 NULL을 반환했을 때 어떤 값을 더 확인해야 하는가?

파일 이름을 삭제한 뒤에도 열린 FD로 읽을 수 있는 이유는?

실습 정리와 제출

mystat으로 종류, inode, 링크 수, 권한, 크기를 확인한다.

링크 생성 전후와 이름 삭제 전후의 결과를 비교한다.

mini_ls에서 디렉토리 끝과 오류를 구분한다.

코드와 실행 로그, 관찰한 차이를 설명하는 글을 제출한다.

파일 내용, 파일 속성, 파일 이름의 관계를 구분해 설명합니다.